CORRIGÉ — SÉRIE 4.2 GestLocation (5 tables + 41 requêtes)

Syntaxe imposée : sans JOIN — FROM ..., ... WHERE ...=... | Dialectes : SQL Server / Access / MySQL

A. Création de la base (5 tables)

Script création + insertion (SQL Server, voir notes MySQL/Access)

```
CREATE DATABASE GestLocation;
GO
USE GestLocation;

CREATE TABLE Marque (
  codeMarque VARCHAR(10) PRIMARY KEY,
  nomMarque  VARCHAR(30) NOT NULL
);

CREATE TABLE Modele (
  codeModele VARCHAR(10) PRIMARY KEY,
  nomModele  VARCHAR(30) NOT NULL,
  codeMarque VARCHAR(10) REFERENCES Marque(codeMarque)
);

CREATE TABLE Voiture (
  matricule   VARCHAR(10) PRIMARY KEY,
  codeMarque  VARCHAR(10) REFERENCES Marque(codeMarque),
  codeModele  VARCHAR(10) REFERENCES Modele(codeModele),
  type        CHAR(1) CHECK (type IN ('C','S','B')),
  prixJour    INT NOT NULL
);

CREATE TABLE Client (
  numCl VARCHAR(10) PRIMARY KEY,
  nom   VARCHAR(40) NOT NULL,
  ville VARCHAR(30)
);

CREATE TABLE Location (
  numLoc    INT IDENTITY(1,1) PRIMARY KEY, -- MySQL: AUTO_INCREMENT
  matricule VARCHAR(10) REFERENCES Voiture(matricule),
  numCl     VARCHAR(10) REFERENCES Client(numCl),
  dateDebut DATETIME NOT NULL,
  nbJours   INT CHECK (nbJours>0),
  assurance VARCHAR(5) CHECK (assurance IN ('TR','S'))
);

-- Jeu d'essai
INSERT INTO Marque VALUES ('M01','Renault'), ('M02','Peugeot'), ('M03','Volkswagen');
INSERT INTO Modele VALUES ('MO01','Clio','M01'), ('MO02','Duster','M01'),
                          ('MO03','Passat','M03'), ('MO04','3008','M02');
INSERT INTO Voiture VALUES ('V01','M01','MO01','C',120), ('V02','M01','MO02','S',180),
                           ('V03','M03','MO03','B',250), ('V04','M02','MO04','S',180);
INSERT INTO Client VALUES ('Cl01','Ahmed Ben Ali','Tunis'), ('Cl02','Leila Mansour','Ariana'),
                          ('Cl03','Sami Trabelsi','Sfax');
INSERT INTO Location (matricule,numCl,dateDebut,nbJours,assurance) VALUES
('V01','Cl02','2022-05-20 09:00',3,'TR'), ('V01','Cl03','2022-04-07 11:00',5,'S'),
('V02','Cl01','2022-06-24 15:00',10,'TR'), ('V03','Cl01','2022-06-24 15:00',10,'TR'),
('V03','Cl03','2022-07-17 08:30',7,'S'),  ('V04','Cl02','2022-06-26 13:30',4,'S');
```

Notes : MySQL → IDENTITY → AUTO_INCREMENT. Access → AutoNumber. Respecter l'ordre de création : Marque → Modele → Voiture → Client → Location (à cause des clés étrangères). Voiture possède 2 FK : une vers Marque et une vers Modele, pour satisfaire la consigne « une voiture doit avoir sa marque et son modèle ».

B. Mises à jour (NOUVEAUTÉS 2026)

2 Mettre à jour l'année de dateDebut à 2026 (garder jour/mois/heure).

SQL Server / MySQL (syntaxe WHERE) :

```
-- SQL Server : ajouter le nombre d'années manquantes (2026 - YEAR(dateDebut))
UPDATE Location
SET dateDebut = DATEADD(year, 2026 - YEAR(dateDebut), dateDebut);

-- Variante avec DATEFROMPARTS (SQL Server 2012+) - conserve l'heure :
UPDATE Location
SET dateDebut = DATEADD(day, DATEDIFF(day, CAST(dateDebut AS date), dateDebut),
                DATEFROMPARTS(2026, MONTH(dateDebut), DAY(dateDebut)));
```

Variantes Access / MySQL :

```
-- MySQL
UPDATE Location SET dateDebut = DATE_ADD(dateDebut, INTERVAL (2026 - YEAR(dateDebut)) YEAR);
-- Access
UPDATE Location SET dateDebut = DateAdd('yyyy', 2026 - Year(dateDebut), dateDebut);
-- Vérification
SELECT numLoc, dateDebut, YEAR(dateDebut) AS annee FROM Location; -- doit afficher 2026 partout
```

→ UPDATE sans WHERE = toutes les lignes. YEAR() extrait l'année, DATEADD(year, n, date) ajoute n années. Si dateDebut=2022-05-20 → 2026-2026-2022=4 → DATEADD(year,4,'2022-05-20')='2026-05-20'. L'heure est conservée.

Résultat attendu : 6 lignes mises à jour. Ex : 20/05/2022 09:00 → 20/05/2026 09:00. Vérifier avec SELECT YEAR(dateDebut).

3 Augmenter de 15% le prix des voitures louées par des clients d'Ariana (UPDATE avec liaison).

SQL Server / MySQL (syntaxe WHERE) :

```
-- Méthode recommandée (sous-requête avec liaison WHERE - sans JOIN)
UPDATE Voiture
SET prixJour = prixJour * 1.15
WHERE matricule IN (
  SELECT L.matricule
  FROM Location L, Client C
  WHERE L.numCl = C.numCl
    AND C.ville = 'Ariana'
);

-- Variante SQL Server avec FROM (toujours sans JOIN, avec WHERE)
UPDATE V
SET prixJour = V.prixJour * 1.15
FROM Voiture V, Location L, Client C
WHERE V.matricule = L.matricule
  AND L.numCl = C.numCl
  AND C.ville = 'Ariana';
```

Variantes Access / MySQL :

```
-- Access (obligatoirement IN, pas de FROM dans UPDATE)
UPDATE Voiture SET prixJour = prixJour * 1.15
WHERE matricule IN (SELECT L.matricule FROM Location AS L, Client AS C WHERE L.numCl=C.numCl AND C.ville='Ariana');
-- Pour annuler : UPDATE Voiture SET prixJour = prixJour / 1.15 WHERE matricule IN (...)
```

→ UPDATE avec sous-requête qui contient une liaison WHERE entre Location et Client. C'est la « mise à jour avec liaison » demandée, mais sans mot-clé JOIN : on utilise FROM ..., ... WHERE ...=... dans la sous-requête. Voitures concernées : V01 (louée par Cl02 Ariana) et V04 (louée par Cl02 Ariana) → 2 voitures → 120→138 et 180→207.

Résultat attendu : 2 lignes mises à jour : V01 120→138 DT, V04 180→207 DT. V02 et V03 inchangées.

C. Requêtes de consultation (20 SELECT - syntaxe WHERE sans JOIN)

4 Afficher tous les clients habitant à Ariana.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT * FROM Client WHERE ville = 'Ariana';
```

→ Filtre simple.

Résultat attendu : Cl02 Leila Mansour Ariana (1 ligne)

5 Donner le nombre de voitures par marque.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT codeMarque, COUNT(*) AS nbVoitures
FROM Voiture
GROUP BY codeMarque;

-- Avec libellé marque :
SELECT M.nomMarque, COUNT(*) AS nbVoitures
FROM Voiture V, Marque M
WHERE V.codeMarque = M.codeMarque
GROUP BY M.nomMarque;
```

→ GROUP BY sur la clé étrangère. Si on veut le nom, liaison WHERE Voiture-Marque nécessaire.

Résultat attendu : Renault (M01):2 Peugeot (M02):1 Volkswagen (M03):1

6 Trouver la (les) voiture(s) la (les) moins chère(s).

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT * FROM Voiture
WHERE prixJour = (SELECT MIN(prixJour) FROM Voiture);

-- Avec marque/modèle :
SELECT V.matricule, M.nomMarque, MO.nomModele, V.prixJour
FROM Voiture V, Marque M, Modele MO
WHERE V.codeMarque = M.codeMarque AND V.codeModele = MO.codeModele
  AND V.prixJour = (SELECT MIN(prixJour) FROM Voiture);
```

→ Sous-requête MIN. Double liaison pour afficher libellés.

Résultat attendu : V01 Renault Clio 120 DT

7 Afficher numLoc, dateDebut, nom client, marque et modèle voiture.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT L.numLoc, L.dateDebut, C.nom, MQ.nomMarque, MO.nomModele
FROM Location L, Client C, Voiture V, Marque MQ, Modele MO
WHERE L.numCl = C.numCl
  AND L.matricule = V.matricule
  AND V.codeMarque = MQ.codeMarque
  AND V.codeModele = MO.codeModele
ORDER BY L.numLoc;
```

→ 4 liaisons WHERE pour relier 5 tables. C'est la nouveauté : on doit passer par Voiture pour remonter à Marque/Modèle.

Résultat attendu : 6 lignes avec détail complet. Ex : 1 | 20/05/2022 | Leila Mansour | Renault | Clio

8.1 Locations dont dateDebut = aujourd'hui.

SQL Server / MySQL (syntaxe WHERE) :

```
-- SQL Server
SELECT L.numLoc, C.nom FROM Location L, Client C
WHERE L.numCl = C.numCl
  AND CONVERT(date, L.dateDebut) = CONVERT(date, GETDATE());
-- MySQL : WHERE DATE(L.dateDebut)=CURDATE()
-- Access : WHERE DateValue(L.dateDebut)=Date()
```

→ Comparer partie DATE sans heure. Après UPDATE Q2, toutes les dates sont en 2026, donc aucune n'est = aujourd'hui (si aujourd'hui=2026, tester avec INSERT GETDATE()).

Résultat attendu : 0 ligne (normal, insérer une ligne TEST avec GETDATE() pour vérifier)

8.2 Locations des 7 derniers jours.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT * FROM Location
WHERE DATEDIFF(day, dateDebut, GETDATE()) BETWEEN 0 AND 7;
-- Avec DATEADD : WHERE dateDebut >= DATEADD(day, -7, GETDATE()) AND dateDebut <= GETDATE()
```

Variantes Access / MySQL :

```
Access : WHERE DATEDIFF('d', dateDebut, NOW()) BETWEEN 0 AND 7
MySQL : WHERE DATEDIFF(CURDATE(), DATE(dateDebut)) BETWEEN 0 AND 7
```

→ DATEDIFF ou DATEADD équivalents.

Résultat attendu : Dépend de la date du jour. Après maj 2026, si on est en octobre 2026, 0 ligne car données mai-juillet 2026 >7j

8.3 Locations il y a plus de 30 jours.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT * FROM Location WHERE DATEDIFF(day, dateDebut, GETDATE()) > 30;
```

→ Même principe >30.

Résultat attendu : 6 lignes (mai-juillet 2026 >30j en oct 2026)

9 Prix moyen par jour de toutes les voitures.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT AVG(prixJour) AS prixMoyen FROM Voiture;
```

→ (120+180+250+180)/4=182.5

Résultat attendu : 182.5 DT

10 Prix moyen par marque.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT M.nomMarque, AVG(V.prixJour) AS prixMoyen
FROM Voiture V, Marque M
WHERE V.codeMarque = M.codeMarque
GROUP BY M.nomMarque;
```

→ Liaison Voiture-Marque + GROUP BY nomMarque.

Résultat attendu : Renault 150, Peugeot 180, Volkswagen 250

11 Lister toutes les locations du client Cl03.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT * FROM Location WHERE numCl='Cl03';
```

→ 2 locations

Résultat attendu : numLoc 2 (V01 5j) et 5 (V03 7j)

12 Montant total facture client Cl03.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT SUM(L.nbJours * V.prixJour) AS factureTotale
FROM Location L, Voiture V
WHERE L.matricule = V.matricule
  AND L.numCl='Cl03';
```

→ 600+1750=2350 DT

Résultat attendu : 2350 DT (ou 2422 si Q3 appliquée : V01=138→690+1750=2440)

13 Clients ayant une location en cours actuellement.

SQL Server / MySQL (syntaxe WHERE) :

```
-- SQL Server
SELECT DISTINCT C.nom FROM Location L, Client C
WHERE L.numCl = C.numCl
  AND GETDATE() BETWEEN L.dateDebut AND DATEADD(day, L.nbJours, L.dateDebut);
```

Variantes Access / MySQL :

```
MySQL : WHERE NOW() BETWEEN L.dateDebut AND DATE_ADD(L.dateDebut, INTERVAL L.nbJours DAY)
Access : WHERE NOW() BETWEEN L.dateDebut AND DateAdd('d', L.nbJours, L.dateDebut)
```

→ Location en cours si aujourd'hui entre debut et debut+nbJours.

Résultat attendu : 0 ligne en oct 2026 (toutes terminées), normal

14 Nombre de clients ayant même assurance que Cl02.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT COUNT(DISTINCT L.numCl) AS nbClients
FROM Location L
WHERE L.assurance IN (SELECT assurance FROM Location WHERE numCl='Cl02');
```

→ Cl02 a TR et S → IN → tous les clients (3).

Résultat attendu : 3

15 Locations dont durée > moyenne.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT * FROM Location WHERE nbJours > (SELECT AVG(nbJours) FROM Location);
```

→ Moyenne 6.5 → >6.5 → 10,10,7

Résultat attendu : 3,4,5

16 Locations dont durée > celle de numLoc=2 (5j).

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT * FROM Location WHERE nbJours > (SELECT nbJours FROM Location WHERE numLoc=2);
```

→ >5 → 10,10,7

Résultat attendu : 3,4,5

17 Nombre de locations pour Citadine (type C).

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT COUNT(*) AS nb
FROM Location L, Voiture V
WHERE L.matricule = V.matricule AND V.type='C';
```

→ V01 seule citadine → 2 locations

Résultat attendu : 2

18 Total jours loués par voiture + marque/modèle.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT V.matricule, MQ.nomMarque, MO.nomModele, SUM(L.nbJours) AS totalJours
FROM Location L, Voiture V, Marque MQ, Modele MO
WHERE L.matricule = V.matricule
  AND V.codeMarque = MQ.codeMarque
  AND V.codeModele = MO.codeModele
GROUP BY V.matricule, MQ.nomMarque, MO.nomModele;
```

Variantes Access / MySQL :

```
-- Simple (sans libellés) : SELECT matricule, SUM(nbJours) FROM Location GROUP BY matricule;
```

→ GROUP BY matricule + liaisons pour libellés.

Résultat attendu : V01 8j Renault Clio, V02 10j Renault Duster, V03 17j VW Passat, V04 4j Peugeot 3008

19 Chiffre d’affaires par voiture et par marque.

SQL Server / MySQL (syntaxe WHERE) :

```
-- Par voiture
SELECT V.matricule, MQ.nomMarque, SUM(L.nbJours * V.prixJour) AS CA
FROM Location L, Voiture V, Marque MQ
WHERE L.matricule=V.matricule AND V.codeMarque=MQ.codeMarque
GROUP BY V.matricule, MQ.nomMarque, V.prixJour;

-- Par marque (BONUS)
SELECT MQ.nomMarque, SUM(L.nbJours * V.prixJour) AS CA_Marque
FROM Location L, Voiture V, Marque MQ
WHERE L.matricule=V.matricule AND V.codeMarque=MQ.codeMarque
GROUP BY MQ.nomMarque;
```

→ SUM(nbJours*prix). Par marque : Renault 2760 (960+1800), VW 4250, Peugeot 720. Total 7730 DT.

Résultat attendu : V03 la plus rentable (4250)

20 Nombre de locations par client et assurance.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT numCl, assurance, COUNT(*) AS nb FROM Location GROUP BY numCl, assurance ORDER BY numCl;
```

→ 6 groupes

Résultat attendu : Cl01-TR2, Cl02-TR1 Cl02-S1, Cl03-S2

21 Voitures triées type croissant, prix décroissant.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT * FROM Voiture ORDER BY type ASC, prixJour DESC;
```

→ B Passat250, C Clio120, S Duster180, S 3008 180

Résultat attendu : 4 lignes triées

22 Clients ≥2 locations.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT numCl, COUNT(*) AS nb FROM Location GROUP BY numCl HAVING COUNT(*)>=2;
```

→ HAVING après GROUP BY

Résultat attendu : Cl01 2, Cl02 2, Cl03 2 (3 clients)

23 Voitures sans location.

SQL Server / MySQL (syntaxe WHERE) :

```
-- NOT IN
SELECT * FROM Voiture WHERE matricule NOT IN (SELECT matricule FROM Location);
-- NOT EXISTS
SELECT * FROM Voiture V WHERE NOT EXISTS (SELECT 1 FROM Location L WHERE L.matricule=V.matricule);
```

→ 2 méthodes sans JOIN. 0 ligne (toutes louées). Ajouter V05 pour tester.

Résultat attendu : 0 ligne

D. Fonctions chaînes & dates complémentaires (Q24-Q35)

24 Libellé complet avec CONCAT : Marque Modèle (Matricule).

SQL Server / MySQL (syntaxe WHERE) :

```
-- SQL Server / MySQL
SELECT CONCAT(M.nomMarque, ' ', MO.nomModele, ' (', V.matricule, ')') AS libelle
FROM Voiture V, Marque M, Modele MO
WHERE V.codeMarque = M.codeMarque AND V.codeModele = MO.codeModele;

-- Access
SELECT M.nomMarque & ' ' & MO.nomModele & ' (' & V.matricule & ')' AS libelle
FROM Voiture AS V, Marque AS M, Modele AS MO
WHERE V.codeMarque = M.codeMarque AND V.codeModele = MO.codeModele;
```

Variantes Access / MySQL :

```
-- MySQL CONCAT_WS : CONCAT_WS(' ', M.nomMarque, MO.nomModele)
-- SQL Server + : M.nomMarque + ' ' + MO.nomModele + ' (' + V.matricule + ')'
```

→ CONCAT assemble plusieurs colonnes. Liaisons V-M et V-MO obligatoires.

Résultat attendu : 4 lignes : Renault Clio (V01), Renault Duster (V02), Volkswagen Passat (V03), Peugeot 3008 (V04)

25 Nom du client + longueur (LENGTH/LEN).

SQL Server / MySQL (syntaxe WHERE) :

```
-- MySQL
SELECT nom, CHAR_LENGTH(nom) AS longueur FROM Client;
-- ou LENGTH(nom) (octets)
-- SQL Server
SELECT nom, LEN(nom) AS longueur FROM Client;
-- Access
SELECT nom, Len(nom) AS longueur FROM Client;
```

→ LENGTH/LEN/CHAR_LENGTH selon SGBD. Compte les caractères, espaces inclus.

Résultat attendu : Ali : 14, Leila Mansour : 13, Sami Trabelsi : 13 (exemples)

26 3 premiers caractères du modèle avec SUBSTRING.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT nomModele, SUBSTRING(nomModele, 1, 3) AS extrait FROM Modele;
-- MySQL : SUBSTRING(nomModele,1,3) ou SUBSTR
-- Access : Mid(nomModele,1,3)
```

→ SUBSTRING(col, début, longueur). 1 = premier caractère.

Résultat attendu : Clio→Cli, Duster→Dus, Passat→Pas, 3008→300

27 3 premières lettres du nom client avec LEFT.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT nom, LEFT(nom, 3) AS debut FROM Client;
-- Access : Left(nom,3)
-- MySQL : LEFT(nom,3)
-- SQL Server : LEFT(nom,3) ou SUBSTRING(nom,1,3)
```

→ LEFT(col, n) = SUBSTRING(col,1,n). Similaire à Q26 mais plus court.

Résultat attendu : Ahmed → Ahm, Leila → Lei, Sami → Sam

28 2 derniers caractères du matricule avec RIGHT.

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT matricule, RIGHT(matricule, 2) AS fin FROM Voiture;
-- Access : Right(matricule,2)
-- MySQL/SQL Server : RIGHT(matricule,2)
```

→ RIGHT(col, n) extrait la fin. Utile pour matricules.

Résultat attendu : V01→01, V02→02, V03→03, V04→04

29 Jour du mois avec DAY() / DAYOFMONTH().

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT numLoc, dateDebut, DAY(dateDebut) AS jour FROM Location;
-- DAY = DAYOFMONTH (synonymes)
-- Access : Day(dateDebut)
-- MySQL : DAY(dateDebut) ou DAYOFMONTH(dateDebut)
```

→ DAY extrait le jour (1-31). Après UPDATE 2026, ex : 20/05/2026 → 20.

Résultat attendu : 1:20, 2:7, 3:24, 4:24, 5:17, 6:26

30 Nom du jour avec DAYNAME().

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT numLoc, DAYNAME(dateDebut) AS jourSemaine FROM Location;
-- MySQL : DAYNAME(dateDebut) → Monday, Tuesday...
-- SQL Server : DATENAME(weekday, dateDebut)
-- Access : WeekdayName(Weekday(dateDebut))
```

→ Retourne le nom en anglais (MySQL) ou selon locale. Pour 20/05/2026 = Vendredi / Friday.

Résultat attendu : Ex : 20/05/2026 → Friday / Vendredi (selon SGBD)

31 Numéro du jour avec DAYOFWEEK().

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT numLoc, DAYOFWEEK(dateDebut) AS numJour FROM Location;
-- MySQL : DAYOFWEEK() → 1=Dimanche 2=Lundi ...7=Samedi
-- SQL Server : DATEPART(weekday, dateDebut)
-- Access : Weekday(dateDebut)
```

→ Convention MySQL : 1=Dimanche. Attention SQL Server peut dépendre de DATEFIRST.

Résultat attendu : 20/05/2026 Vendredi → 6 (MySQL)

32 Jour dans l'année avec DAYOFYEAR().

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT numLoc, DAYOFYEAR(dateDebut) AS jourAnnee FROM Location;
-- MySQL : DAYOFYEAR(dateDebut) → 1-366
-- SQL Server : DATEPART(dayofyear, dateDebut)
-- Access : DatePart('y', dateDebut)
```

→ 1er janvier =1, 31 décembre =365/366. Ex : 20 mai = 140ème jour environ.

Résultat attendu : 07/04 → 97, 20/05 → 140, 24/06 → 175

33 Mois avec MONTH().

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT numLoc, MONTH(dateDebut) AS mois FROM Location;
-- Tous SGBD : MONTH(date) → 1-12
```

→ Extrait le mois. Utile pour filtrer WHERE MONTH(dateDebut)=6 (juin).

Résultat attendu : 1:5 (mai), 2:4 (avril), 3:6, 4:6, 5:7, 6:6

34 Année avec YEAR().

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT numLoc, YEAR(dateDebut) AS annee FROM Location;
```

→ Après UPDATE Q2, toutes les années =2026. Vérif : GROUP BY YEAR(dateDebut).

Résultat attendu : 6 lignes avec 2026

35 Date système avec NOW() / GETDATE().

SQL Server / MySQL (syntaxe WHERE) :

```
-- MySQL
SELECT NOW() AS maintenant; -- ou CURRENT_TIMESTAMP
-- SQL Server
SELECT GETDATE() AS maintenant;
-- Access
SELECT NOW() AS maintenant;
-- Résultat : 2026-10-19 14:30:00 (selon horloge serveur)
```

→ NOW/GETDATE sans paramètre. Pour date seule : CURDATE() (MySQL) / CONVERT(date,GETDATE()) (SQL Server) / Date() (Access).

Résultat attendu : Une ligne avec date/heure système

E. Autre UPDATE + 2 DELETE avec liaison (Q36-Q38)

36 Mettre à jour ville à 'Ariana' pour clients ayant loué Volkswagen (liaison).

SQL Server / MySQL (syntaxe WHERE) :

```
UPDATE Client SET ville = 'Ariana'
WHERE numCl IN (
  SELECT L.numCl
  FROM Location L, Voiture V, Marque M
  WHERE L.matricule = V.matricule
    AND V.codeMarque = M.codeMarque
    AND M.nomMarque = 'Volkswagen'
);
-- Variante avec EXISTS
UPDATE Client SET ville='Ariana'
WHERE EXISTS (SELECT 1 FROM Location L, Voiture V, Marque M
  WHERE L.matricule=V.matricule AND V.codeMarque=M.codeMarque
    AND M.nomMarque='Volkswagen' AND L.numCl=Client.numCl);
```

→ UPDATE avec sous-requête contenant 2 liaisons (L-V et V-M). Clients concernés : Cl01 (a loué V03 VW Passat) et Cl03 (a loué V03) → 2 clients passent à Ariana.

Résultat attendu : 2 lignes mises à jour : Cl01 Tunis→Ariana, Cl03 Sfax→Ariana. Vérif : SELECT * FROM Client WHERE ville='Ariana' → 3 lignes.

37 Supprimer les locations <5j de voitures Citadine (liaison).

SQL Server / MySQL (syntaxe WHERE) :

```
DELETE FROM Location
WHERE nbJours < 5
  AND matricule IN (
    SELECT matricule FROM Voiture WHERE type='C'
  );
-- Variante avec EXISTS + liaison
DELETE FROM Location
WHERE nbJours < 5 AND EXISTS (
  SELECT 1 FROM Voiture V
  WHERE V.matricule = Location.matricule AND V.type='C'
);
```

→ DELETE avec sous-requête et liaison Location-Voiture. Seule V01 est Citadine (C) avec nbJours 3 → 1 ligne supprimée (numLoc 1). Attention : DELETE sans WHERE supprime tout !

Résultat attendu : 1 ligne supprimée : numLoc 1 (V01, 3j, Citadine). Restent 5 locations. Annuler avec ROLLBACK si transaction.

38 Supprimer les clients sans aucune location (liaison).

SQL Server / MySQL (syntaxe WHERE) :

```
DELETE FROM Client
WHERE numCl NOT IN (SELECT numCl FROM Location);
-- ou NOT EXISTS (plus sûr avec NULL)
DELETE FROM Client
WHERE NOT EXISTS (
  SELECT 1 FROM Location L WHERE L.numCl = Client.numCl
);
```

→ DELETE avec liaison Client-Location via NOT IN / NOT EXISTS. Avec le jeu d'essai, tous les clients ont une location → 0 ligne supprimée. Pour tester : INSERT Client Cl04 sans location → 1 ligne supprimée.

Résultat attendu : 0 ligne (cas normal). Après INSERT Cl04 'Test Sousse' → 1 ligne supprimée.

F. Analyse : requête et/ou résultat → rôle (corrigé)

39 Rôle : SELECT M.nomMarque, COUNT(*) ... GROUP BY M.nomMarque

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT M.nomMarque, COUNT(*) AS nb
FROM Voiture V, Marque M
WHERE V.codeMarque = M.codeMarque
GROUP BY M.nomMarque;
```

→ Rôle : Cette requête affiche le nombre de voitures par marque. Elle compte les voitures groupées par marque grâce à la liaison Voiture-Marque (V.codeMarque=M.codeMarque) et GROUP BY.

Résultat attendu : Renault | 2, Peugeot | 1, Volkswagen | 1. Réponse attendue : 'Donner le nombre de voitures pour chaque marque'.

40 Rôle : SELECT nom, LENGTH(nom) FROM Client WHERE ville='Ariana';

SQL Server / MySQL (syntaxe WHERE) :

```
SELECT nom, LENGTH(nom) AS longueur
FROM Client
WHERE ville='Ariana';
-- MySQL : CHAR_LENGTH(nom) | SQL Server : LEN(nom) | Access : Len(nom)
```

→ Rôle : Elle affiche le nom des clients d'Ariana avec la longueur de leur nom (nombre de caractères). WHERE filtre Ariana, LENGTH calcule.

Résultat attendu : Leila Mansour | 13 caractères. Réponse attendue : 'Afficher le nom et sa longueur pour les clients d'Ariana'.

41 Rôle : DELETE FROM Location WHERE nbJours<5 AND matricule IN (SELECT matricule FROM Voiture WHERE type='C')

SQL Server / MySQL (syntaxe WHERE) :

```
DELETE FROM Location
WHERE nbJours < 5
  AND matricule IN (SELECT matricule FROM Voiture WHERE type='C');
```

→ Rôle : Supprime les locations de courte durée (<5 jours) concernant des voitures Citadine (type C). Liaison Location-Voiture via sous-requête IN. Une seule ligne concernée : numLoc 1 (V01 Renault Clio, 3 jours). Après exécution, il reste 5 locations.

Résultat attendu : 1 ligne supprimée : V01 3j Citadine. Rôle = 'Supprimer les petites locations des citadines'.

BONUS CORRIGÉS :

```
-- CA total par marque (exploite nouvelles tables)
SELECT MQ.nomMarque, SUM(L.nbJours*V.prixJour) AS CA
FROM Location L, Voiture V, Marque MQ
WHERE L.matricule=V.matricule AND V.codeMarque=MQ.codeMarque
GROUP BY MQ.nomMarque ORDER BY CA DESC; -- VW 4250 > Renault 2760 > Peugeot 720

-- Vue complète 5 tables
CREATE VIEW VueFactureComplete AS
SELECT L.numLoc, C.nom, C.ville, MQ.nomMarque, MO.nomModele, V.type, V.prixJour,
       L.dateDebut, DATEADD(day, L.nbJours, L.dateDebut) AS dateFin,
       L.nbJours * V.prixJour AS montant
FROM Location L, Client C, Voiture V, Marque MQ, Modele MO
WHERE L.numCl=C.numCl AND L.matricule=V.matricule
  AND V.codeMarque=MQ.codeMarque AND V.codeModele=MO.codeModele;

-- Après UPDATE Q2, vérifier : SELECT YEAR(dateDebut), COUNT(*) FROM Location GROUP BY YEAR(dateDebut);
```

Conseils pédagogiques & pièges

- UPDATE 2026 : ne pas oublier YEAR() et conserver l'heure → DATEADD(year, 2026-YEAR(date), date) et non UPDATE sans WHERE avec date fixe.
- UPDATE avec liaison : penser à la sous-requête IN + liaison WHERE Location.numCl=Client.numCl.
- Nouvelle structure 5 tables : pour afficher marque/modèle, il faut 2 liaisons : Voiture.codeMarque=Marque.codeMarque ET Voiture.codeModele=Modele.codeModele.
- Piège Q7/Q18 : oublier une des 4 liaisons → produit cartésien.
- Piège Q20 : NOT IN échoue si sous-requête a NULL → préférer NOT EXISTS.